ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/321737981

The Docker Security Playground: A hands-on approach to the study of network security

Conference Paper · September 2017

DOI: 10.1109/IPTCOMM.2017.8169747

CITATIONS
22

READS
2,446

2 authors:

Gaetano Perrone
University of Naples Federico II
30 PUBLICATIONS **166** CITATIONS

SEE PROFILE

Simon Pietro Romano
University of Naples Federico II
189 PUBLICATIONS **1,446** CITATIONS

SEE PROFILE

All content following this page was uploaded by Simon Pietro Romano on 28 December 2017.

The user has requested enhancement of the downloaded file.

The Docker Security Playground: a hands-on approach to the study of Network Security

G. Perrone*, S. P. Romano*
*University of Napoli Federico II,
Department of Electrical Engineering and Information Technology,
Via Claudio 21, 80125 Napoli, Italy
ga.perrone@studenti.unina.it,spromano@unina.it

***Abstract*—This paper presents the design and implementation of the Docker Security Playground (DSP), an architecture leveraging a microservices-based approach in order to build complex network infrastructures specifically tailored to the study of network security. DSP has been conceived at the outset as a tool for learning network security with a hands-on approach. A number of security labs have been already realized and made available in a public repository. The paper discusses how such labs can be fruitfully exploited by students, as well as presents the Application Programming Interface offered to programmers interested in the implementation of new labs.**

I. Introduction

Network security involves complex network infrastructures, so a thorough methodology is necessary to locate all possible vulnerabilities. A common vulnerability assessment approach follows all typical "hacking" steps: (i) *footprinting*: to gather useful information about a potential target; (ii) *scanning* and *enumeration*: to look for vulnerable services on the target network; (iii) *exploit*: to gain internal network access by exploiting found vulnerabilities in a target-network host; (iv) *post-exploit*: to attack other target-network hosts, do privilege escalation, install Remote Access Tools (RAT) and backdoors, in order to fully compromise the network.

To support these operations many tools have been developed. Docker [1] offers a flexible way to have all these services installed in one's own PC through lightweight containers. This is a great innovation that is linked to modern microservices architectures, as well as to modern agile development processes that use Continuous Integration (CI) approaches: a Docker image that exposes a network security service can be pushed into a public registry and can be updated on local PCs by simply pulling it. Docker Security Playground (DSP) takes advantage of this and provides network security tools that can be used in its virtual labs.

Virtualization indeed represents an unprecedented way to optimize resources and to have different environments on a single machine. Its main goals are resource optimization, load balancing, high availability and multi-tenancy. For network security researchers, virtualization has quickly become an essential way to generate network infrastructures, since it enables the following set of functionality:

- to learn network security basics;
- to simulate different network security scenarios by avoiding legal issues;
- to test new vulnerabilities without breaking real network infrastructures;
- to study viruses, worms, and all threats that could potentially destroy a real-world environment;
- to cap training and experiment costs.

Many of the above mentioned operations (scanning, exploit) are illegal, so the main difficulty that arises when trying to learn about network security is to do so without falling in trouble with law. To solve this issue, many virtual network security labs have been developed by security teams. These typically offer a remote infrastructure that uses classical virtualization approaches to simulate vulnerable networks. Anyway, most such solutions do not take advantage of the flexibility allowed by containers. Docker Security Playground offers a way to do so. The limited footprint of Docker images allows to run all labs on a COTS PC with no need for a remote infrastructure: users only need Docker and git to synchronize with the provided *DockerLabs* and subsequently have a fully fledged management platform available for experimenting with network security. Docker allows to create a microservices-based platform composed of labs that are developed by using an Infrastructure as Code (IaC) approach. IaC is a modern paradigm allowing to configure and manage infrastructures through code, without the need to manage complex physical hardware configurations or to use interactive tools. This approach makes it easy to manage servers' replication and configuration changes, by reducing the risk of configuration errors. In our context IaC is a flexible way to configure advanced **vulnerable** network infrastructures. Delving a bit more into the details, DSP uses *docker-compose* [2], a technology that allows to define complex network infrastructures by simply writing a text file. DSP also defines an image standard called *Docker Image Wrapper* that simplifies the automation of some operations.

Microservices are an architectural pattern that allows to build modular and scalable applications by decoupling all the functionality in a set of loosely coupled, collaborating services. This approach enhances modularity, scalability and makes it easier to manage complex applications. Through Docker technology it is possible to have lightweight services

978-1-5386-1322-1/17/$31.00 ©2017 IEEE

on one's own PC without the cost of a full virtualization. Docker Security Playground takes advantage of this feature to provide users with a full microservices-based framework for the implementation of attack scenarios in virtualized network infrastructures. Lab designers can create their advanced network labs by using images created by other users (image designers). Students can use these labs to learn each phase of a vulnerability assessment process.

In summary, Docker Security Playground is a platform that provides:

- A Docker-Compose management platform to start and stop labs;
- An advanced Web GUI to create new labs through a drag-and-drop interface that generates docker-compose files;
- *Docker Image Wrapper*, a standard for docker images, that helps the DSP application understand docker-images functionality and other aspects such as exposed ports, description, etc.;
- A set of Docker Vulnerable Images and Docker Tools Images that can be used to create new labs;
- A repository of labs, that will be daily updated by lab designers.

The following section will present current state of the art in the field of network security tools and learning assets. Then, we will briefly discuss DSP design and implementation, while also providing information about the Docker Image Wrapper standard. Finally one simple yet complete example scenario will be described, before drawing some conclusions and identifying directions of future work.

II. State of the art

A lot of programs and tools have been created and released by the community with two main purposes: to simplify the penetration testing process and to help beginners in learning network security. We can classify these in: (i) Penetration Testing Tools and Distros; (ii) Learning Penetration Testing Software; (iii) Learning Penetration Testing Environments.

Metasploit [7] is the most used penetration testing software. It contains a rich exploit database allowing to simulate real-world attacks on a network to train a company's security team about security issues. Metasploit also includes other tools such as *nmap* and *NeXpose* and it can perform a thorough vulnerability assessment. It provides advanced post-exploit elements such as *Meterpreter* that enables users to control the screen of a device by using VNC and to browse, upload and download files. Metasploit is an open source project. Its exploits are written in Ruby, so it is fully configurable.

When talking about web vulnerabilities, specific scanners have been developed: a common one is Nikto [4], a generic web scanner that reveals a lot of common web server issues, such as information disclosure, misconfiguration problems, vulnerable installed software, etc. There are also CMS specific vulnerability scanners, such as WPScan [5] for Wordpress and JoomlaAttack [6] for Joomla. Specific tools to perform a semiautomatic analysis of web vulnerabilities also exist. These tools integrate a lot of features, such as web proxy, customization of GET and POST requests, request repeaters, web spidering, fuzzing data sender. A very well-known commercial tool is *Burp Suite* [8] by PortSwigger. An amazing open source tool is *Zed Attack Proxy* (from OWASP project) [9], [14].

Advanced Penetration Testing Linux Distributions have been developed. The most well known is *Kali* (successor of Backtrack) [10], a Debian GNU/Linux distribution created by Offensive Security that contains a large collection of security and forensic tools, such as Wireshark, John the Ripper, aircrack-ng. An ARM version of Kali is also available, running on Raspberry Pis.

Vulnerable software is also available online. On the so-called *vulnhub* site [11] there is an unlimited list of vulnerable VMs and vulnerable software to run in order to practice. Every day a new challenge is published on the site. Metasploitable and DamnVulnerableLinux [12] are very vulnerable Linux distributions that can be used to test common service exploits, and privilege escalation. These VMs are commonly used to learn the basics of Metasploit.

Online vulnerability databases are an excellent resource to learn about common vulnerabilities. *exploit-db* [13] for example gives a vulnerability description, provides source code to exploit the vulnerability and sometimes also gives source code of vulnerable applications, so you can compile it on a virtual machine and try to exploit the vulnerable service.

About Web Application vulnerabilities, OWASP project [14] is an online community which creates freely-available articles, methodologies, documentation, tools, and technologies in the field of web application security. OWASP offers a lot of training materials and a completely vulnerable target website, called *WebGoat*, which can be used to understand most common web vulnerabilities, such as SQLi (SQL injection), XSS (Cross Site Scripting), CSRF (Cross Site Request Forgery). Other important website distributions are *bodgeit* and *DamnVulnerableWebApps*.

A lot of security resources have been developed to learn network security. Offsensive Security offers professional training by providing virtual penetration testing labs and by offering different types of Security Certifications. Metasploit is used to exploit all basic labs vulnerabilities. Metasploit might require some minor modifications to work with different environments, so for a penetration tester it is important to learn about buffer overflow basics. A good online environment is *Over the wire*[1], that provides different 'CTF' (Capture The Flag) challenges where you have 'ssh' (secure shell) access to the lab and you are asked to exploit some vulnerable services to win the challenge.

With the growing tendency of container-based virtualization, many developers are working to convert network security tools and vulnerability services into Docker images.

Docker provides a microservice architecture, so each tool can be converted into a specific Docker image (for example nmap can be implemented as a Docker nmap image). Docker

[1] http://overthewire.org/wargames/narnia/

images also give full support to vulnerable services: currently WebGoat, DVWA, bodgeit and many other applications are also made available as Docker images. On DockerHub registry there is a growing tendency to implement Docker vulnerable images, because of the flexibility that this technology allows to users. Docker Security Playground leverages all these images and offers a platform to effectively use them.

On the infrastructure side, there is little evidence on-line about the use of Docker-Compose as a complete security labs platform. The only things that can be currently found on-line are preliminary uses of Vagrant to simulate vulnerable network environments.

The innovation of Docker Security Playground is to use Docker-Compose to generate vulnerable IaC labs, by offering a flexible and lightweight platform to create, manage and simulate virtual vulnerable environments by using docker images, without the cost of a complete virtualization such as VBox or VmWare. Docker Security Playground also provides a docker images standard (Docker Image Wrapper) to automate the virtual development of labs. Currently many images have been developed, such as a full Kali image distribution with *shellinabox* support (that can be used to navigate the OS via browser), metasploit, and all most important network security tools discussed previously. With DSP it is possible to implement a real enterprise scenario like the ones commercially provided by Offensive Security and to simulate hacker attacks, in order to practice with network security. Key components of such scenarios are network elements like a router base image with iptables support to simulate firewalling scenarios and a *LAMP* (Linux, Apache, MySQL and PHP) web server to simulate php vulnerable web services

III. The Docker Security Playground: an overview

In this section we will present the DSP architecture, by analyzing some important use cases, such as the creation of a lab.

DSP is an application that offers a microservices-based framework to create, manage and use network security labs. Each lab helps the user learn a specific network security topic. Network security Labs are grouped into repositories. DSP can manage multiple repositories; each repository is a set of labs that are remotely stored. All repositories are saved in a root application directory that is chosen by the user upon installation of the system.

More precisely, when the user installs DSP, a default network security repository (named *DSP_Projects*) is downloaded from the remote repository and installed in the root application directory. Then, a user network security repository is created. All repository images are stored and located inside the repository root directory and built when the repository is imported. Anyway, images used by the official repository are stored in DockerRegistry hub under the ad hoc created *dockersecplayground* organization[2].

[2]https://hub.docker.com/u/dockersecplayground/

Each lab has descriptive information that helps users understand the goal of the lab, including the name of the lab, a brief description, its main goal and the related solution.

A lab infrastructure is defined by a docker-compose file. Though, the user does not write a docker-compose file directly. The application indeed provides a graphic docker-compose editor: the user draws the containers in a white canvas, then saves the network and a docker-compose file is generated by the application. The graphic description also contains information about actions and files to copy inside the container.

A network is composed of several containers and subnets. When users create containers, they select a docker image for each of them. If the image uses the *Docker Wrapper Image* standard (that will be defined later) it exposes some actions that users can select during the network creation process. Other things that the user can add to containers are the files to copy inside the deployed container instance. All these docker commands will be set during the lab network creation and they will be executed after the lab is started and all its containers are active.

A user perspective of DSP is sketched in Fig. 2.

App developers modify the DSP application. DSP Lab makers use the application to create new labs and these labs can update the DSP_Projects official repository. DSP_Student users learn network security by using the DSP_Projects lab through the DSP application. Labs use Docker Wrapper Images that expose special actions, so there is another user, called *DockerImageDeveloper*, who creates new images that are added to the list of images used by a repository.

A top level view of the DSP application is reported in Fig. 3.

It is a client-server application: all user requests are sent through the DSP user interface layer and are forwarded to the DSP server. Fig. 4 shows a graphic representation of user's interaction with the mentioned user interface layer.

The labs in a repository use Docker Images, so each lab has docker-image dependencies.

Each lab has a description file, a network file, a labels file and a docker-compose.yml file. The description file is created when the user creates the lab for the first time. The network file is created when the user draws a network and saves it. The docker-compose file is created by DSP through the network file information when the user saves the network description.

A. Docker Wrapper Image

There are two problems that led us to the creation of the Docker Wrapper Image standard.

During lab creation the user creates new containers. Each container has an associated image. This image can offer a lot of actions, for example to create new users, to start a sql server or a web server. These actions can be executed after that the container has started, and can even be parameterized (e.g., to create two different users in a container one might want to use the "create user" action with different input arguments). A second issue arises when the user creates the network and chooses the images for the container. Upon that occasion, she/he has to know information about the selected image, the

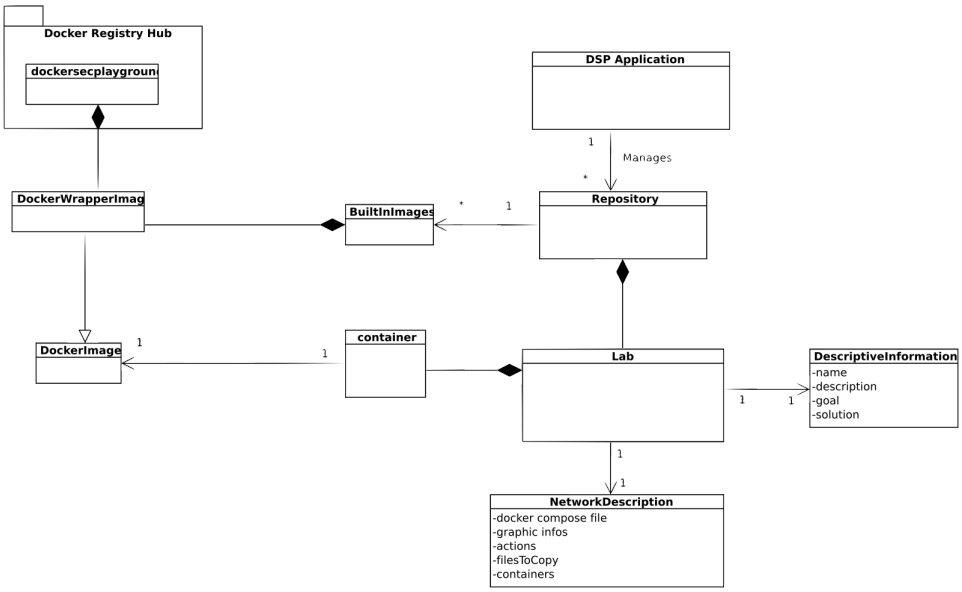

Fig. 1: Top Level View of DSP application

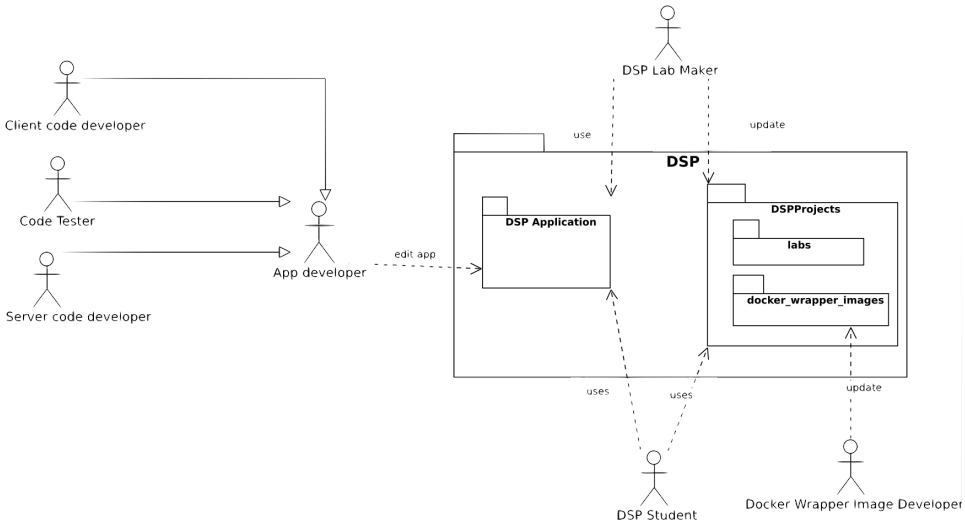

Fig. 2: DSP User view

actions and port services that the image exposes (a web server for example exposes port 80, an FTP server port 21, etc.).

Docker image creation is done by writing a Dockerfile. In Dockerfile you can set all the operations that will be executed when the container is run, as well as the software that has to be installed by default in a docker container.

Dockerfile has a well-defined syntax with many options. In your Dockerfile you can define labels that are metadata that can be useful to describe the image. Docker Wrapper Image defines a standard notation for the labels, so the DSP application can write Docker image labels, understand and parse them and forward them to the user. A label is represented as a key-value pair. An example of a Dockerfile that uses Docker Image Wrapper notation is reported in Fig. 5.

In this Dockerfile there is the image definition on the top, followed by instructions for updating the apk repository and downloading some apk packages. Then, some scripts are loaded in the image. In order to allow DSP to expose these scripts to users during image selection (upon graphical creation of the docker-compose file), we make use of the labels notation. Such labels are compliant with the proposed Docker Wrapper Image notation and actually allow the DSP application to read and interpret them. They concern features like the specific type of image (server, router, etc.), the need for special privileges at the container level (e.g., to make proper use of the loaded scripts), the need for undertaking a well-defined set of actions.

B. Summary of DSP Application Use Cases

A list of DSP application uses cases is the following.

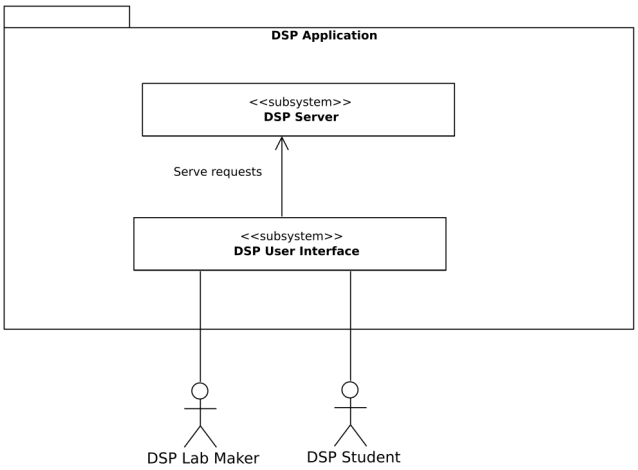

Fig. 3: DSP Application view

- *Create new lab*: User creates a new lab, by adding information like name, description, goal and solution. User can also select and add labels from the list of repository labels. When the user submits the operation:
  - a new lab directory is created in the user repository directory;
  - a new entry in table state file is created with NO_NETWORK state;
- *Add network description to lab*: Once created the lab, user goes in the graphic editor and creates a graphic network description by using the GUI. DSP saves all information about containers and subnets and generates the docker-compose file. The lab entry in the state table changes from NO_NETWORK to STOPPED.
- *Import Lab*: The user wants to import a lab from a repository (such as DSP_Projects repo) in local user repository. A new entry in state table is created with repository equal to user repository and name equal to lab name. A new lab directory is created in the user repository directory, all the files that containers use are copied in the .data directory of the user repository. All the used labels in imported lab are copied in the user repository set of labels.
- *Copy Lab*: The user copies a lab into his local user repository. A new lab directory is created in local user repository with same information, except for the name (that is changed by adding a trailer versioning part). A new entry in state table is added with new copied lab name and local user repository.
- *Create a new label*: The user creates a new label in his local user repository. He selects the name of label, a description and a color for the label. A new label is added to the list of labels of user's local repository.
- *Edit a label*: The user edits an existing label. He chooses a label by name, changes information (can change color, name, description) and submits the operation. The user cannot edit the label if a label with same name is present in the set of repository labels.
- *Delete a label*: The user deletes an existing label in user's repository: he chooses a label by name and DSP deletes the label from the list of user repository labels. The deleted label might be associated with some lab, so DSP checks each lab in user repository to find if the deleted label has been associated with the lab. When a lab is found, the deleted label is removed from the list of lab labels.
- *Start lab*: The user starts a lab. The docker-compose command is started in lab directory (with 'docker-compose up' operation). When all the containers are active, the entry of the lab in the state table changes from STOPPED to RUNNING and all the copies and action operations are executed. If some errors occur, they are shown on the user interface, the lab is stopped (if running) and the lab entry in the state table goes back to STOPPED.
- *Stop lab*: The user stops a lab. DSP executes 'docker-compose down' command, all containers are destroyed and the lab entry in the state table changes from RUNNING to STOPPED.
- *Add a new file to user local repository*: User adds a new file to .data directory of his local repository. This file is made available to graphic network interface when the user wants to copy files inside a container.

After creating new labs, users can graphically define the underlying network. A graphic network is simply a set of customized containers and subnets. You can define different subnets in your network description: each subnet has a 24 bit netmask and has a name and a color (to distinguish between different networks). When you load new subnets these can be selected during the creation of a container.

A typical lab creation process is:

1) create the subnets that you need;
2) create and customize containers;
3) link containers to created subnets and add to graphic network description;
4) save the graphic network definition;
5) let DSP create and show the resulting docker-compose file.

Each container has a name, an image and different docker-compose options;

When you select a container you can do different things:

- select the name of the image;
- link the container to a specific network;
- select docker-compose options for a container (caps_add, volumes, etc.);
- select the actions to be performed inside the container;
- select the files to copy inside the container.

With respect to actions, they are selected when you have chosen an image for a container. When you create a new container you have to select an image. If this image follows the Docker Wrapper Image standard, it exposes some actions (actions are scripts loaded inside the image). The associated labels of these scripts are parsed by DSP and are showed on

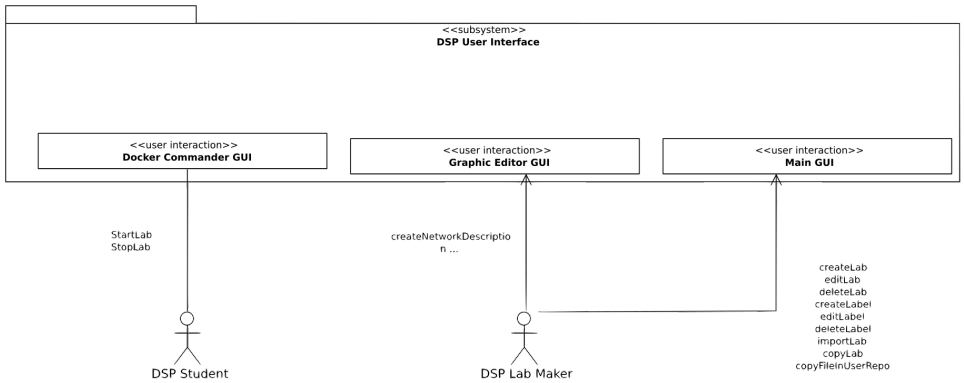

Fig. 4: DSP User interaction

```
FROM alpine:latest
ENV ENV="/root/.ashrc"
RUN apk --update add bash && \
 echo "alias sh=bash" > "$ENV"

COPY adduser.sh /adduser.sh
COPY setgw.sh /setgw.sh
COPY addctf.sh /addctf.sh
COPY exec.sh /exec.sh

LABEL \
      type="host" \
      caps_add="NET_ADMIN" \
      actions.exec.command="/exec.sh " \
      actions.exec.description="Execute a command in the container" \
      actions.exec.args.command.val="mkdir hello" \
      actions.exec.args.command.type="text" \

      actions.adduser.command="/adduser.sh" \
      actions.adduser.description="Add a new username : <name> <password> " \
      actions.adduser.args.name.val="user" \
      actions.adduser.args.name.type="text" \
      actions.adduser.args.name.rule.pattern="^[a-zA-Z0-9_-]+$" \
      actions.adduser.args.password.val="password" \
      actions.adduser.args.password.type="text" \
      actions.adduser.args.password.rule.pattern="^[a-zA-Z0-9_-]+$" \

        actions.addctf.command="/addctf.sh" \
      actions.addctf.description="Add a ctf in /home/<username> directory inside a secret file" \
      actions.addctf.args.username.val="" \
      actions.addctf.args.username.type="text" \
      actions.addctf.args.username.rule.pattern="^[a-zA-Z0-9_-]+$" \
      actions.addctf.args.ctf.val="" \
      actions.addctf.args.ctf.type="text" \

      actions.setgw.command="/setgw.sh" \
      actions.setgw.description="Set default gateway  <name container gateway> " \
      actions.setgw.args.gateway.val="" \
      actions.setgw.args.gateway.type="text" \
      actions.setgw.args.gateway.rule.pattern="^[a-zA-Z0-9_-]+$" \
```

Fig. 5: Lab states

the GUI, so the user can select different actions, set the action arguments and load the action in the container that will be loaded in the network definition. All the selected actions will be executed just after the lab is started.

It is also possible to select the priority of the actions: each action has a priority that is a number (between 1 and 100). Sometimes you have to set a specific order for the actions (for example, you cannot load a SQL database inside a container if you have not yet started the SQL server). The priority number defines the execution order of the actions: a lower number has a greater priority, so it is executed earlier.

One more thing that one can do with a container is loading files. Obviously you cannot choose an absolute path inside your PC: each user has a local repository that can share with other users if he synchronizes his repo remotely. So the files to copy inside a container will be selected from a data directory inside the root DSP application directory. These files are loaded in the .data folder by the user beforehand. A selectable option when you copy a file is the "after action" flag. Sometimes you want to do actions before copying a file (for example if you want to load a secret key inside a user directory the user must exist, so you have to create the user first). Other times you want to do actions after that you copy a file (for example if you want to execute a script that you have loaded in the container, you have to load the script first, then you can run the "execute the script" action). Similarly, when you add a file to a container you can set the "after action" flag, so the related copy operation is performed after the actions. If you do not set the "after action" flag, the copy operation is carried out before any other action.

IV. A SAMPLE ATTACK SCENARIO WITH DSP

A number of attack scenarios have been implemented with DSP. An up-to-date, thoroughly documented list of such scenarios can be found on the project's gitlab repository, which is publicly available (under the MIT license) at the following URL: http://gitlab.comics.unina.it/NS-Thesis/DockerSecurityPlayground_1.

For the sake of conciseness, we are going to present here just one of them, which is nonetheless deemed to be representative of the variegated set of functionality made available by the DSP platform. The scenario in question is a typical SQL injection attack, called *TopolinuxAgency*.

A. TopolinuxAgency: playing with SQL injection

In this scenario, network topology is quite simple: a Kali machine that you can use to exploit SQL injection with the sqlmap tool. There is a single relevant Docker Image that is a web server

1) Description: In this scenario an SQL injection attack will be explained. Network topology is simple: a kali machine that you can use to exploit sql injection with sqlmap. There is a single relevant Docker Image that is a web server vulnerable to SQL injection (Fig. 6).

2) Goal: Open web server on http://localhost:9000/topolinux_agency/ and discover Jack Barlock's password!

3) Solution: The vulnerable web server consists in a single page, as shown in Fig. 7.

The password to be discovered is *'ForeverAlone1988'*.

When you open the web browser you will be prompted with a form with a search engine. If you try to insert any character you will notice an SQL error. If you insert a number, the result

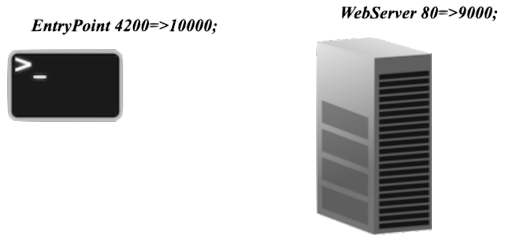

Fig. 6: TopolinuxAgency Network

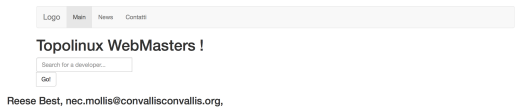

Fig. 7: TopolinuxAgency Site

will be the id of the user who developed the application. If one tries to enter a single quote as an input in the query form, the following error will be reported:

```
You have an error in your SQL syntax; check the manual that
corresponds to your MySQL server version for the right
syntax to use near ''' at line 5
```

This is a clear SQL INJECTION vulnerability, associated with the absence of checks on user input parameters. We can hence try and exploit it. In order to do so, open Kali 'shellinabox' tool, and find the web server in the internal network by using the ping command (to find the IP address) and nmap command with -p 80 option to find if the host is running a web server:

```
kali@283d23a3ca18:~$ ping 172.23.0.2
PING 172.23.0.2 (172.23.0.2) 56(84) bytes of data.64 bytes
from 172.23.0.2: icmp_seq=1 ttl=64 time=0.206 ms
kali@283d23a3ca18:~$ nmap 172.23.0.2 -p 80

Starting Nmap 7.40 ( https://nmap.org ) at 2017-05-16
Nmap scan report for topolinux_WebServer_1
Host is up (0.00031s latency).
Not shown: 999 closed ports
PORT    STATE SERVICE
80/tcp open  http
```

As expected, the web server is listening on port 80 on address 172.23.0.2. By looking at the previous server response, the form url was the following:

```
http://localhost:9000/topolinux_agency/?developer=1
```

So let's try to exploit the SQL injection vulnerability with *sqlmap*:

1) Let's check the vulnerability and find the database:

```
sqlmap -u "http://172.23.0.2/topolinux_agency/?
developer=1" --dbs
[09:11:23] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Apache 2.4.7, PHP 5.5.9
back-end DBMS: MySQL >= 5.5
[09:11:23] [INFO] fetching database names
available databases [5]:
[*] exampleDB
[*] information_schema
[*] mysql
[*] performance_schema
[*] web_db
```

2) We have just discovered the list of databases, while also realizing that the web server in question is a vulnerable one. Now let's try to dump the list of tables:

```
kali@283d23a3ca18:~$ sqlmap -u
"http://172.23.0.2/topolinux_agency?developer=1"
--dbms Mysql -D web_db --tables

[09:16:55] [INFO] testing MySQL
[09:16:55] [INFO] confirming MySQL
[09:16:55] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Apache 2.4.7, PHP 5.5.9
back-end DBMS: MySQL >= 5.0.0
[09:16:55] [INFO] fetching tables for database:'web_db'
Database: web_db
[1 table]
+-----------+
| usernames |
+-----------+
```

3) There is only one table. Let's dump all the users inside the table:

```
sqlmap -u http://172.23.0.2/topolinux_agency?developer=
1 --dbms Mysql -D web_db -T usernames --dump

[INFO] fetching columns for table 'usernames'
[INFO] fetching entries for table 'usernames'
[INFO] analyzing table for possible password hashes
Database: web_db
Table: usernames
[101 entries]
| id | name | email | password |
| 1 | Sonya | aliquam@luctuset.com | KFB40ENA3JK |
| 2 | Griffi| urna@massalobortis.org | NXA58YLZ8TR |
| 3 | Shea | lobortis@sagittis.co.uk | OWD53WAT2AO |
| 4 | Colette | dis@Morbi.net | ANS71YZB7AC |
| 5 | Geoffrey | feugiat@luctus.edu | NSY63TKP7MU |
|... |
| 101| JackBarlock | jack@rete.com | ForeverAlone1988 |
```

The last dump shows that we were eventually able to retrieve the password, as required when setting the goals of the lab.

V. Conclusions

In this paper we presented the Docker Security Playground, a microservices-based architecture for the seamless creation and management of virtual networking labs. DSP has been conceived at the outset as a tool for facilitating the study of network security and has to be considered as a permanent work in progress, to be continuously fed with new scenarios and equipped with new functionality.

For the future we intend to refine the system by also implementing further (advanced) docker compose options. We are also going to look after the implementation of ad hoc 'suspend/resume' techniques to be associated with a lab's lifecycle management. One further milestone will be the realization of a number of attack scenarios to be made available in the public project's repository.

Most importantly, we intend to attract interest towards our architecture from the international research community. Our goal is to make DSP become as rich as possible in terms of offered scenarios and use cases, so as to contribute to the

evangelization associated with network security, as well as with the variegated set of facets and implications that it entails.

ACKNOWLEDGMENT

This work was partially funded by the European Space Agency, within the framework of project SHINE (Secure Hybrid In Network caching Environment), ESA Contract N.: 4000118273/16/NL/CLP.

REFERENCES

[1] Docker web site.
https://www.docker.com/
[2] Docker compose.
https://docs.docker.com/compose/
[3] Security tools. NMAP .
https://nmap.org
[4] Security tools. Nikto .
http://sectools.org/tool/nikto/
[5] WPScan Team. wpscan .
https://wpscan.org
[6] OWASP . Joomla Security Scanner .
https://www.owasp.org/index.php/Category:OWASP_Joomla_Vulnerability_Scanner_Project
[7] Rapid7 . Metasploit .
https://www.metasploit.com
[8] Portswigger . Burp Suite . https://portswigger.net/burp/
[9] OWASP . Zed Attack Proxy .
https://www.owasp.org/index.php/ZAP
[10] Offensive Security . Kali Linux .
https://www.kali.org
[11] Vulnhub site .
https://www.vulnhub.com
[12] DistroWatch. Damn Vulnerable Linux .
https://distrowatch.com/table.php?distribution=dvl
[13] Offensive Security. Offensive Security?s Exploit Database Archive .
https://www.exploit-db.com
[14] OWASP Team. OWASP .
https://www.owasp.org/index.php/Main_Page

View publication stats